MOSAIC-ML: A Separated-Evidence Framework for Static Android Application Risk Analysis

Astik Sarkar
Department of Computer Science
[Institution Name], [City], [Country]
astiksar20@gmail.com

***Abstract*—**Android risk-analysis tools commonly reduce several unrelated indicators to a single numeric verdict, which conceals both the origin and the reliability of each indicator. This paper presents MOSAIC-ML, a static analysis framework that keeps four signals separate: a machine-learning malware classifier, a permission-weight risk score, a CVE/NVD correlation lookup, and a host-intrusion rule module. Three alternative classifiers—logistic regression, linear support vector machine, and a 30-tree random forest—were compared on the MH-100K dataset under one deduplicated, stratified protocol; nine configurations were scored on validation data only and the test split was evaluated once. Class-weighted logistic regression was selected on balanced accuracy (0.9631 on validation) and achieved 0.9569 balanced accuracy and 0.9427 malware recall on the test split. A single-feature control using only the count of active features reached 0.9450 validation accuracy, showing that accuracy alone is an unsafe selection metric on this dataset: the selected model exceeds the control by 2.3 accuracy points but by 16.6 points of malware recall. The permission score attains a measured ROC-AUC of 0.7734 and returns an explicit undefined state, rather than zero, when no weighted permission is requested. Every reported figure derives from executed, reproducible experiments.

***Keywords*—**Android security; static analysis; machine learning; permission risk; vulnerability correlation; evidence separation

Note*—This document is a structured sample draft that fixes the reporting format and ties every measured value to an executed experiment. The submitted version will be rewritten in the author's own prose. All numerical results were produced by scripts/tune_models.py and the analysis scripts in the accompanying repository, and were reproduced independently in the accompanying notebook.*

I. INTRODUCTION

A. Background

Android remains the dominant mobile operating system and, consequently, the principal target of mobile malware. Its permission model and open distribution channels allow applications to request access to sensitive resources, and users are rarely positioned to evaluate whether a requested capability is proportionate to an application's stated function [1], [2]. Static analysis of the application package has therefore become the standard first line of automated assessment, because it requires neither execution nor instrumentation.

Machine-learning classifiers trained on static features have reported high detection rates across a decade of literature [3], [4]. In parallel, probabilistic scoring of requested permissions has been proposed as a complementary, interpretable signal [5], [6]. Vulnerability databases such as the National Vulnerability Database supply a third, entirely independent category of evidence concerning published weaknesses in software components.

B. Motivation

These evidence categories measure different things. A classifier output expresses resemblance to training data. A permission score expresses a statistical property of requested capabilities. A CVSS value expresses the severity of a published vulnerability and says nothing about whether a particular application is affected. Despite this, many prototype systems combine them into one weighted number, which removes the ability of a reader to determine which signal produced a verdict or how reliable that signal was for the sample under analysis.

C. Problem Statement

Existing integrated Android risk tools conflate heterogeneous evidence into a single score without reporting the measurement reliability of each constituent signal, and without distinguishing an absence of evidence from evidence of low risk. This prevents informed interpretation of the output and permits an unreliable constituent signal to propagate silently into a final verdict.

D. Contributions

- A separated-evidence architecture in which four static signals are computed, reported and interpreted independently rather than aggregated into one number.
- A measured comparison of three classifiers on MH-100K under a single deduplicated protocol, with every configuration and a single-feature control reported, not only the best result.
- An implementation of the permission-weight function of Ogwara et al. adapted to MH-100K, with weights fitted on training data only and an explicit undefined state.
- A per-scan feature-coverage measure quantifying how much of the trained model could be populated from a given application.

II. RELATED WORK AND GAP ANALYSIS

A. Learning-Based Static Detection

DREBIN [3] established broad static feature extraction with a linear classifier and explainable output. Li et al. [4] identified significant permission subsets to reduce dimensionality, Feizollah et al. [7] showed that declared intents carry information beyond permissions, and Idrees et al. [8] combined permissions and intents in an ensemble. Among recent work, Singh and Singh [9] compare logistic regression, linear SVM, random forest, Bernoulli naive Bayes and XGBoost on manifest permissions with engineered summaries including a dangerous-permission score; this overlap means a permission-based three-classifier comparison is not, by itself, a novel contribution. Huang et al. [10] model suspicious subgraphs for interpretable detection.

B. Permission Risk Scoring

Alshehri et al. [11] estimate risk from permission co-occurrence across benign and malicious corpora. Rashidi et al. [12] adopt a hidden Markov formulation over observed resource usage. Xiao et al. [13] derive a minimum necessary permission set and treat the excess as over-privilege. Ogwara et al. [5] define a probabilistic weight per dangerous permission from its relative prevalence in each class and an application score as the mean weight of the permissions it requests.

C. Evaluation Methodology and Drift

Molina-Coronado et al. [14] re-evaluated ten Android detectors and identified duplicated samples, label attribution, class imbalance, evasion and application evolution as sources of optimistic results. Muzaffar et al. [15] reimplemented 18 feature-based studies on 124,000 applications and report that simpler models often outperform more complex ones. Temporal drift is examined by the LAMDA benchmark [16], spanning 2013–2025 (excluding 2015), and by online learning with drift detection [17]. These findings motivate the deduplication and control-baseline procedures adopted here. AndroCom [18] addresses vulnerability detection in source code, which is distinct from malware classification and from the keyword correlation used here.

Reported results in this recent work include random-forest accuracy of 81.6% on permissions alone [9], accuracy up to 96.01% with online learning on API call graphs [17], an F1 score of 97.14% on 14,520 applications [10], and accuracy above 98% across reimplemented feature-based studies [15]. Being from different datasets, labels and metrics, they are context, not a ranking.

D. Comparative Analysis

TABLE I. COMPARISON OF REPRESENTATIVE APPROACHES

Work	Focus	Limitation Addressed Here
Arp et al. [3]	Static ML detection	Single verdict; no per-sample reliability measure
Li et al. [4]	Permission selection	Permissions only; no vulnerability evidence
Ogwara et al. [5]	Permission risk + ensemble	Thresholds specific to its own corpus
Xiao et al. [13]	Over-privilege	Assumes developers over-request consistently
Molina-C. [14]	Evaluation critique	Identifies inflation; does not build a system
Singh et al. [9]	Permission ML, 5 models	Single verdict; no coverage or vulnerability evidence
This work	**Separated evidence**	**Reports each signal with its own reliability**

E. Research Gap

No reviewed system reports, for the individual application under analysis, how much of its trained model could be populated from that application, nor does any distinguish an unanswered vulnerability query from a negative result. Both omissions allow a confident-looking verdict to be produced from inputs that do not support one. MOSAIC-ML addresses this gap.

III. PROBLEM FORMULATION

A. System Model

Let a denote an Android application package. Static analysis yields permissions P(a), declared intents I(a), and invoked framework methods C(a). Let V denote the feature vocabulary of a trained model, |V| = 24,833, partitioned into blocks V_P, V_I and V_C of sizes 166, 250 and 24,417.

B. Formal Definition

The mapping φ(a) → {0,1}^|V| produces a binary feature vector. The coverage of a is the proportion of the vocabulary populated from the application:

$$cov(a) = | \varphi(a) \cap V | \; / \; | V | \qquad (1)$$

The objective is an evidence tuple E(a) = ⟨m, R, X, H, cov⟩, where m is the classifier output, R the permission risk score, X the vulnerability correlation result, H the host-rule result and cov the coverage. Each element must be independently interpretable and carry its own reliability statement rather than being reduced to a scalar.

C. Assumptions and Constraints

Applications are analysed statically and never executed; a vulnerability keyword match is a candidate, not a determination of affectedness; model parameters, permission weights and thresholds are fitted on training data only; and an absence of evidence is reported as undefined, never as zero risk.

IV. PROPOSED METHODOLOGY

A. Dataset and Labelling

Experiments use MH-100K [19], a static-feature corpus of Android applications collected from AndroZoo [20] over 2010–2022. The dataset article reports 101,975 samples; the released feature matrix and its label file each contain 101,934 rows, so the 41-sample difference originates in the distributed files and not in our processing. The matrix has 24,833 binary features and was verified row by row against the label file by SHA-256.

Two candidate label sets in the distribution disagree on 10,505 rows. The array supplied with the feature matrix corresponds exactly to the rule vt_detection ≥ 4 (at least four VirusTotal engines), at 100% agreement, and was adopted as a

stated project decision, since detection-count thresholds are a known source of label disagreement [14].

TABLE II. DATASET COMPOSITION

Property	Count	Share
Rows as distributed	101,934	100%
Benign (vt < 4)	92,134	90.39%
Malicious (vt ≥ 4)	9,800	9.61%
Duplicate feature rows removed	43,358	42.54%
Unique rows retained	58,576	100%
Malicious after deduplication	7,671	13.10%

Duplicates are rows with identical sets of active feature indices; 1,024 of them carry a label different from the retained row and form an irreducible error floor. After deduplication a classifier predicting "benign" for every sample would score 0.869 accuracy.

B. Experimental Protocol

Rows with identical active-feature sets were collapsed to one representative before splitting, so that copies of one application cannot occupy both training and test partitions. The 58,576 unique rows were split by stratified sampling into training (41,003), validation (8,786) and test (8,787) partitions with a fixed seed of 42. All configurations were scored on validation data only. The selected configuration was refitted on training plus validation data and evaluated once on the test partition. Fig. 1 shows the complete training and evaluation flow, including the integrity check that guards against a mis-read feature matrix.

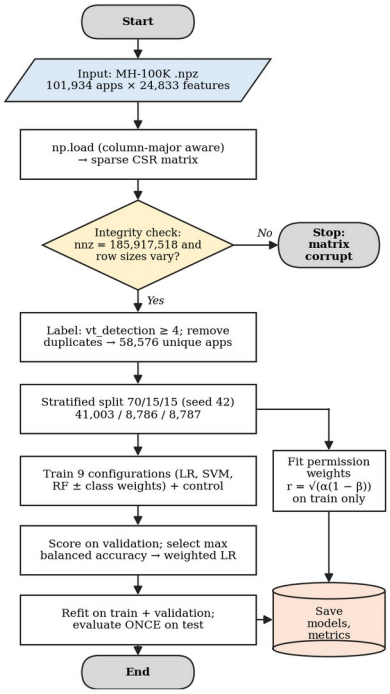


Fig. 1. Training and evaluation flowchart. Oval: start/end; parallelogram: input; rectangle: process; diamond: decision; cylinder: stored artifacts. Permission weights are fitted on the training split only, in parallel with classifier training.

C. Classifiers and Configurations

Three alternative classifiers were trained with scikit-learn [21] on sparse input: logistic regression, a linear SVM, and a random forest fixed at 30 trees. Because malicious samples are the minority class, each was trained both unweighted and with class weights inversely proportional to class frequency, giving nine configurations in total (Table III). Selection used balanced accuracy, chosen before results were seen, because accuracy rewards neglect of the minority class.

A control model was added that receives one input—the number of active features in a sample—and ignores which features are active. It quantifies how much the 24,833-dimensional representation contributes beyond a trivially available property.

D. Permission Risk Scoring

The function of Ogwara et al. [5] was implemented for the same fifteen dangerous permissions and refitted on MH-100K. For permission i, α_i is its prevalence among malicious training applications and β_i its prevalence among benign training applications:

$$r_i = \sqrt{(\alpha_i - \alpha_i \beta_i)} = \sqrt{(\alpha_i (1 - \beta_i))} \qquad (2)$$

For application a, $\lambda(a,i) = 1$ if a requests permission i, and $k(a) = \Sigma_i \lambda(a,i)$. The application score is the mean weight over the permissions requested:

$$R(a) = \Sigma_i \lambda(a,i) \cdot r_i \;/\; k(a), \quad k(a) > 0 \qquad (3)$$

The term $(1 - \beta_i)$ penalises ubiquity: a permission requested by nearly all applications receives a low weight regardless of its prevalence among malicious samples. When $k(a) = 0$ the score is undefined, and the implementation returns a null value with the status insufficient permission evidence rather than zero. Weights were computed on the 41,003 training rows only.

E. Vulnerability Correlation

Component strings extracted from the package are submitted as keyword queries to the NVD API. Three outcomes are reported distinctly: records returned, zero records returned, and lookup failed; collapsing these lets a tool report an absence of vulnerabilities it never established. Severity is retained as the published CVSS value and version [22] and is not transformed into a probability.

F. Performance Metrics

With malware as the positive class and TP, TN, FP, FN the confusion-matrix counts, the following are reported:

$$Prec = TP/(TP+FP), \; Rec = TP/(TP+FN), \; Spec = TN/(TN+FP) \quad (4)$$

$$F1 = 2 \cdot Prec \cdot Rec/(Prec+Rec), \; BalAcc = (Rec+Spec)/2 \quad (5)$$

$$MCC = (TP \cdot TN - FP \cdot FN)/\sqrt{[(TP+FP)(TP+FN)(TN+FP)(TN+FN)]} \quad (6)$$

ROC-AUC and PR-AUC use continuous scores. Balanced accuracy and MCC are robust to the 13.1% malware prevalence.

V. SYSTEM ARCHITECTURE

The prototype is a FastAPI service that also serves a React dashboard, so one process provides the whole application. Fig. 2 shows the scan flow. Coverage is computed before classification so that the classifier output can be qualified by it. A scan summary and the SHA-256 of the package are stored in SQLite, or PostgreSQL when configured; the package itself is not

retained. A prototype weighted aggregate is retained for continuity with earlier work and labelled deprecated.

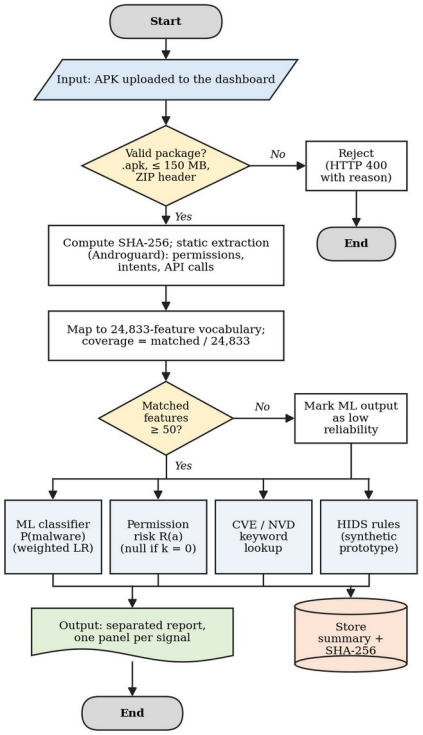


Fig. 2. Scan flowchart of the deployed system. An invalid upload is rejected before parsing; low coverage does not stop the scan but attaches a reliability flag to the classifier output; the four evidence streams are computed independently and reported separately (document symbol) and summarised in the database (cylinder).

VI. RESULTS AND DISCUSSION

A. Classifier Comparison

All results were produced with Python 3.10, scikit-learn 1.6.1, NumPy 1.26.4 and SciPy 1.15.3, and were reproduced to four decimal places by an independent notebook run. Of the nine configurations, class weighting improved validation balanced accuracy for every classifier (logistic regression 0.9598 to 0.9631, linear SVM 0.9573 to 0.9612, random forest 0.9493 to 0.9504), chiefly by raising malware recall at the cost of a higher false-positive rate; the regularisation strength of logistic regression had almost no effect (0.9630–0.9631). Table III reports the best configuration of each classifier. Logistic regression attained the highest balanced accuracy and malware recall. The random forest attained the highest accuracy, F1 and ROC-AUC and the lowest false-positive rate, so the selection is a trade-off rather than a uniform ranking; under the pre-declared criterion, class-weighted logistic regression was selected. Its saved model occupies 0.15 MB against 2.81 MB for the random forest.

TABLE III. VALIDATION PERFORMANCE (DEDUPLICATED, N = 8,786)

Model	Acc.	Bal. Acc.	Mal. Rec.	Mal. F1	FPR
Logistic Regr.	**0.9680**	**0.9631**	**0.9565**	**0.8867**	**.0303**
Linear SVM	0.9660	0.9612	0.9548	0.8802	.0323
Random Forest	0.9735	0.9504	0.9191	0.9007	.0183
Control (count)	*0.9450*	*0.8794*	*0.7904*	*0.7901*	*.0317*

Best configuration per classifier; all three are class-weighted (logistic regression C = 1, linear SVM C = 0.1, random forest 30 trees). ROC-AUC: 0.9793, 0.9767 and 0.9823 respectively. The control uses a single input: the count of active features.

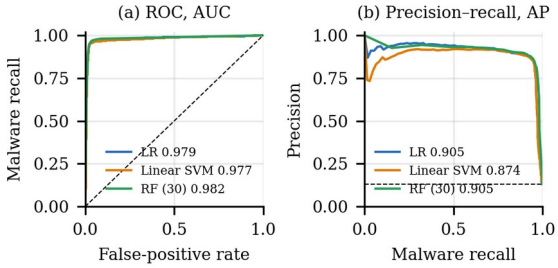


Fig. 3. ROC (a) and precision–recall (b) curves on the validation split. Dashed lines: random ranking; in (b) this equals the malware prevalence, 0.131.

B. The Accuracy Trap

A model that ignores which features are present, and uses only how many, reaches 0.9450 accuracy. The selected model exceeds it by 2.3 accuracy points but by 8.4 points of balanced accuracy and 16.6 points of malware recall. Malicious samples carry a median of 42 active features against 2,082 for benign samples, and 78.3% of malicious samples have fewer than 100 active features against 2.9% of benign samples. This pattern is consistent with failed API-call extraction on packed or obfuscated packages, so a classifier can partly learn that extraction yielded little rather than that behaviour is malicious. Accuracy alone would substantially overstate the contribution of the feature representation (Fig. 4).

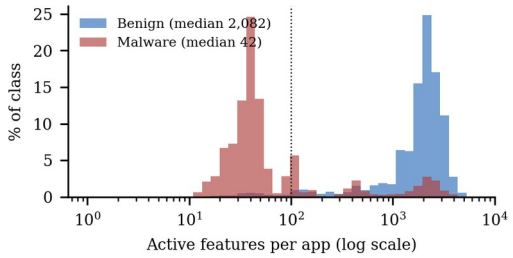


Fig. 4. Active features per application in the raw corpus, as a percentage of each class. The dotted line marks 100 features.

C. Final Evaluation

The selected configuration, refitted on training and validation data, was evaluated once on the test partition: accuracy 0.9675, balanced accuracy 0.9569, malware precision 0.8314, malware recall 0.9427, F1 0.8836, false-positive rate 0.0288, ROC-AUC 0.9785 and PR-AUC 0.8978. The confusion matrix comprises 7,416 true negatives, 220 false positives, 66 false negatives and 1,085 true positives: 66 malicious samples were missed and 220 benign applications were flagged. Table IV lists all test metrics, and Fig. 5 the confusion matrix.

TABLE IV. SELECTED MODEL, SINGLE TEST EVALUATION (N = 8,787)

Metric	Value	Metric	Value

Accuracy	0.9675	Specificity	0.9712
Balanced accuracy	0.9569	False-positive rate	0.0288
Precision (malware)	0.8314	False-negative rate	0.0573
Recall (malware)	0.9427	Neg. predictive value	0.9912
F1 (malware)	0.8836	MCC	0.8670
ROC-AUC	0.9785	Cohen's kappa	0.8647
PR-AUC	0.8978	Inference / app	0.0075 ms

Specificity, NPV, MCC and kappa are derived arithmetically from the recorded confusion matrix. Inference time is batch prediction of the saved model over the test split on the development machine and excludes static extraction.

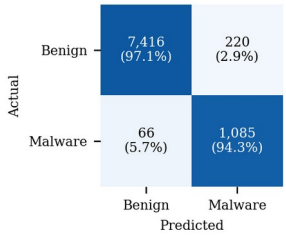


Fig. 5. Test-split confusion matrix of class-weighted logistic regression (counts and share of each actual class).

D. Permission Risk

READ_PHONE_STATE received the highest weight (0.7947), followed by GET_TASKS (0.7871) and MOUNT_UNMOUNT_FILESYSTEMS (0.7793); SEND_SMS received the lowest (0.3399). READ_PHONE_STATE is also the highest-ranked permission reported by Ogwara et al. (0.8460) on a different corpus. WRITE_EXTERNAL_STORAGE illustrates the weighting: it is requested by 97.3% of malicious training applications (α = 0.9732) but also by 72.4% of benign ones (β = 0.7239), which reduces its weight to 0.5183.

On the test partition the score alone attains ROC-AUC 0.7734 and PR-AUC 0.2804 and is defined for 84.9% of samples. It is therefore a weak standalone discriminator and is not presented as a malware probability. Fig. 6 shows all fifteen weights.

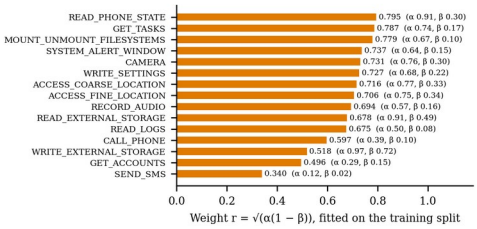


Fig. 6. Permission weights fitted on the training split, with each permission's prevalence among malicious (α) and benign (β) training applications.

E. The Two Analyses Together

On the test partition the permission score and the classifier output correlate weakly (Pearson r = 0.30), so they carry different information. Fig. 7 groups test samples by which analysis flags them, using a classifier threshold of 0.5 and the 85th percentile of the permission score; these descriptive thresholds were not tuned. When both flag a sample it is malicious 87.0% of the time, against 83.1% when only the classifier flags it, a gain of 3.8 points. When only the permission score flags a sample it is malicious 1.4% of the time. The permission score can therefore corroborate a classifier flag but must never overrule the classifier; its role is a transparent signal that remains computable when coverage is too low to trust the classifier.

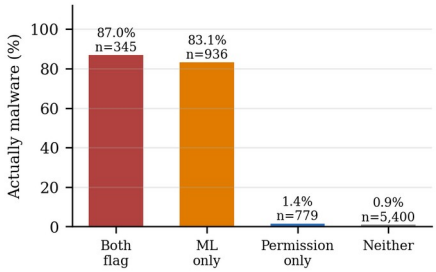


Fig. 7. Share of test samples that are malicious, grouped by which analysis flags them.

F. Worked Examples and System Output

Table V traces three applications through the evidence streams. Example A is a real package (com.github.yeriomin.smsscheduler, 0.07 MB) uploaded to the running system; static extraction took 0.08–0.53 s over three runs, the first including library warm-up. Examples B and C are the first malicious and first benign applications of the test split in split order, chosen without regard to their scores; they are dataset rows, so CVE and host-rule evidence do not apply.

TABLE V. WORKED EXAMPLES (ACTUAL SYSTEM OUTPUT)

Evidence	A: uploaded APK	B: test malware	C: test benign
Ground truth	not labelled	Malicious	Benign
Features P / I / A	5 / 1 / 91	10 / 2 / 8	2 / 0 / 1,245
Coverage (≥ 50?)	97: sufficient	20: low	1,247: sufficient
P(malware), label	0.0045, benign	0.9775, malware	0.0002, benign
Weighted perms	2 of 15	3 of 15	0 of 15
R(a)	0.5673	0.6537	undefined (k = 0)
CVE / NVD	5 queries, 0 records	n/a	n/a

P / I / A: permission, intent and API-call features matched. Example A ran on the author's machine with network access; NVD answered all five queries with zero records, which is reported as "zero records", not as "no vulnerabilities".

The arithmetic is direct. For A, which requests READ_PHONE_STATE and SEND_SMS, R = (0.7947 + 0.3399) / 2 = 0.5673; for B, which requests ACCESS_COARSE_LOCATION, WRITE_EXTERNAL_STORAGE and WRITE_SETTINGS, R = (0.7160 + 0.5183 + 0.7267) / 3 = 0.6537; for C no weighted permission is requested, so R is undefined rather than zero. Both scores lie below the benign class mean (0.6461), which illustrates why the score is not used to overrule the classifier. Example B also illustrates the coverage gate: it carries only 20 active features, is classified correctly, yet is reported with a low-reliability flag, because its verdict rests on exactly the sparse-input pattern identified in Section VI-B. For A, the deprecated prototype aggregate is 0.40 × 0.45 + 0.40 × 30.67 + 0.20 × 0 =

12.45, of which 12.27 derives from synthetic host-rule data; reporting the signals separately makes this visible.

VII. LIMITATIONS AND THREATS TO VALIDITY

- Results derive from a single stratified split, without repeated runs, confidence intervals or probability calibration.
- The test partition was also used in an earlier development run of the unweighted configuration (balanced accuracy 0.9515); selection here used validation data only, but the partition is not pristine across the project's history.
- The corpus spans 2010–2022; generalisation to contemporary applications is untested, and temporal drift is a documented source of optimism [14], [16].
- Under the adopted threshold, 10,992 samples flagged by one to three engines and 172 samples without a scan result are labelled benign.
- Host-intrusion telemetry is synthetic; NVD matches are unverified keyword candidates; the retained aggregate is an unvalidated, deprecated formula.

VIII. CONCLUSION AND FUTURE WORK

MOSAIC-ML reports heterogeneous static evidence separately rather than aggregating it. Three classifiers were compared on MH-100K under a deduplicated protocol with nine configurations scored on validation data; class-weighted logistic regression was selected and achieved 0.9569 balanced accuracy and 0.9427 malware recall in a single test evaluation. A single-feature control showed that accuracy alone is an unsafe selection metric on this dataset, and the refitted permission score proved a useful corroborating, but not standalone, signal.

Future work will replace the deprecated aggregate with an explicit evidence policy whose thresholds are selected on validation data and frozen before evaluation, consistent with established risk-assessment practice [23]; evaluate temporal generalisation on a longitudinal benchmark [16]; substitute device-derived telemetry for the synthetic host-intrusion events; and strengthen vulnerability correlation with version evidence so that candidate matches can be confirmed.

REFERENCES

[1] P. O. Rai, Android Application Security Essentials. Birmingham, U.K.: Packt Publishing, 2013.

[2] B. Bonné, S. T. Peddinti, I. Bilogrevic, and N. Taft, "Exploring decision making with Android's runtime permission dialogs using in-context surveys," in Proc. 13th Symp. Usable Privacy and Security (SOUPS), Santa Clara, CA, USA, 2017.

[3] D. Arp, M. Spreitzenbarth, M. Hübner, H. Gascon, and K. Rieck, "DREBIN: Effective and explainable detection of Android malware in your pocket," in Proc. 21st Annu. Netw. Distrib. Syst. Security Symp. (NDSS), San Diego, CA, USA, 2014.

[4] J. Li, L. Sun, Q. Yan, Z. Li, W. Srisa-An, and H. Ye, "Significant permission identification for machine-learning-based Android malware detection," IEEE Trans. Ind. Informat., vol. 14, no. 7, pp. 3216–3225, 2018.

[5] N. O. Ogwara, K. Petrova, M. L. Yang, and S. G. MacDonell, "A risk assessment framework for mobile apps in mobile cloud computing environments," Future Internet, vol. 16, no. 8, art. 271, 2024.

[6] Y. Wang, J. Zheng, S. Sun, and S. Mukkamala, "Quantitative security risk assessment of Android permissions and applications," in Proc. 27th IFIP WG 11.3 Conf. Data Appl. Security Privacy, Newark, NJ, USA, 2013.

[7] A. Feizollah, N. B. Anuar, R. Salleh, G. Suarez-Tangil, and S. Furnell, "AndroDialysis: Analysis of Android intent effectiveness in malware detection," Comput. Security, vol. 65, pp. 121–134, 2017.

[8] F. Idrees, M. Rajarajan, M. Conti, T. M. Chen, and Y. Rahulamathavan, "PIndroid: A novel Android malware detection system using ensemble learning methods," Comput. Security, vol. 68, pp. 36–46, 2017.

[9] A. Singh and K. Singh, "Permission-based Android malware detection," Indian J. Comput. Sci., vol. 10, no. 5, pp. 33–52, 2025, doi: 10.17010/ijcs/2025/v10/i5/175885.

[10] H. Huang, W. Huang, and F. Jiang, "GSIDroid: A suspicious subgraph-driven and interpretable Android malware detection system," Sensors, vol. 25, no. 13, art. 4116, 2025, doi: 10.3390/s25134116.

[11] A. Alshehri, P. Marcinek, A. Alzahrani, H. Alshahrani, and H. Fu, "PUREDroid: Permission usage and risk estimation for Android applications," in Proc. 3rd Int. Conf. Inf. Syst. Data Mining (ICISDM), Houston, TX, USA, 2019.

[12] B. Rashidi, C. Fung, and E. Bertino, "Android resource usage risk assessment using hidden Markov model and online learning," Comput. Security, vol. 65, pp. 90–107, 2017.

[13] J. Xiao, S. Chen, Q. He, Z. Feng, and X. Xue, "An Android application risk evaluation framework based on minimum permission set identification," J. Syst. Softw., vol. 163, art. 110533, 2020.

[14] B. Molina-Coronado, U. Mori, A. Mendiburu, and J. Miguel-Alonso, "Towards a fair comparison and realistic evaluation framework of Android malware detectors based on static analysis and machine learning," Comput. Security, vol. 124, art. 102996, 2023.

[15] A. Muzaffar, H. Ragab Hassen, H. Zantout, and M. A. Lones, "Reassessing feature-based Android malware detection in a contemporary context," PLoS ONE, vol. 21, no. 1, art. e0341013, 2026, doi: 10.1371/journal.pone.0341013.

[16] M. A. Haque, I. Hossain, M. M. Kamol, M. J. Alam, S. K. Amalapuram, S. Talukder, and M. S. Rahman, "LAMDA: A longitudinal Android malware benchmark for concept drift analysis," in Proc. Int. Conf. Learn. Represent. (ICLR), 2026; arXiv:2505.18551.

[17] M. D. Hussain and A. Muzaffar, "Online learning-based Android malware detection using API call graphs and drift detection: A comparative study," in Proc. AAAI Symp. Ser., vol. 6, no. 1, pp. 87–89, 2025, doi: 10.1609/aaaiss.v6i1.36036.

[18] K. E. Arikan and E. N. Yılmaz, "AndroCom: A real-world Android applications' vulnerability dataset to assist with automatically detecting vulnerabilities," Appl. Sci., vol. 15, no. 5, 2025, doi: 10.3390/app15052665.

[19] H. Bragança, V. Rocha, L. Barcellos, E. Souto, D. Kreutz, and E. Feitosa, "Android malware detection with MH-100K: An innovative dataset for advanced research," Data in Brief, vol. 51, art. 109750, 2023.

[20] K. Allix, T. F. Bissyandé, J. Klein, and Y. Le Traon, "AndroZoo: Collecting millions of Android apps for the research community," in Proc. IEEE/ACM 13th Work. Conf. Mining Softw. Repositories (MSR), Austin, TX, USA, 2016, pp. 468–471.

[21] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[22] Forum of Incident Response and Security Teams, "Common Vulnerability Scoring System version 4.0: Specification document," FIRST, 2023. [Online]. Available: https://www.first.org/cvss/v4.0/specification-document

[23] National Institute of Standards and Technology, "Guide for conducting risk assessments," NIST Special Publication 800-30 Rev. 1, Gaithersburg, MD, USA, 2012.